

TP02 - Practice 3: Dependency Injection and Circular Dependency (NestJS)

Objective

This exercise demonstrates how Dependency Injection (DI) works in NestJS across services and modules, and how to handle circular dependencies.

Main goals:

1. Build provider chains (service uses another service).
2. Understand module visibility rules (imports and exports).
3. Reproduce a circular dependency scenario.
4. Fix circular dependency using design refactoring with an abstraction token.

What This Project Implements

Notification flow

- When an order is created, the app emits an order event and notifies through NotificationsService.
- When a receipt is created or updated, the app notifies through NotificationsService.

DI design used in this project

- OrdersService depends on NotificationsService.
- ReceiptsService depends on NotificationsService.
- NotificationsService depends on EVENT_PUBLISHER token (not on OrdersService).
- CoreModule provides EVENT_PUBLISHER.

This avoids circular dependency between OrdersService and NotificationsService.

Module Overview

- AppModule: root composition, TypeORM setup, feature module registration.
- OrdersModule: order endpoint and microservice client event emission.
- ReceiptsModule: CRUD for receipts using TypeORM repository.
- NotificationsModule: central notification service used by Orders and Receipts.
- CoreModule: shared token-based provider (EVENT_PUBLISHER).

Circular Dependency Notes

Problem (concept)

A circular dependency happens when:

- Service A depends on Service B, and
- Service B depends on Service A.

At module level, a circle appears when Module A imports Module B and Module B imports Module A.

Better fix (used here)

Instead of making NotificationsService call OrdersService directly, NotificationsService uses an abstraction token:

- EVENT_PUBLISHER token is injected.
- CoreModule provides the concrete publish implementation.

Benefits:

- No circular dependency needed.
- Cleaner boundaries.
- Easier to test and replace implementations.

API Endpoints

Root

- GET / -> returns Hello World

Orders

- POST /orders -> creates an order event and sends notification

Receipts

- GET /receipts
- GET /receipts/:id
- POST /receipts
- PATCH /receipts/:id
- DELETE /receipts/:id

Note: receipts endpoints are protected by API key guard.

Validation and Interceptors

- Global ValidationPipe is enabled:
 - whitelist: true
 - forbidNonWhitelisted: true
 - transform: true
- Global LoggingInterceptor is enabled.

Tech Stack

- NestJS
- TypeORM
- SQLite
- class-validator
- class-transformer

Project Setup

```
npm install
```

Run

```
# build
npm run build

# start
npm run start

# development watch mode
npm run start:dev
```

Test

```
npm run test
npm run test:e2e
npm run test:cov
```

Suggested Demo Steps (for class/lab)

1. Start app.
2. Call POST /orders and observe notification publishing logs.
3. Call POST /receipts and PATCH /receipts/:id and observe notification publishing logs.
4. Explain that notifications use a token-based abstraction to prevent direct service cycles.

Key Learning Outcome

In NestJS, DI is module-graph based. Good architecture uses abstraction and exports/imports correctly to keep dependencies one-directional and maintainable.